MODULE 3 · LECTURE 14-b

CS 6530 Applied Cryptography

Authenticated Diffie–Hellman

Module 3 – Public-Key Cryptography

AUTHENTICATED DIFFIE–HELLMAN

How Does Alice Know the DH Value Really Came From Bob?

DH Key Agreement + **Authentication** → **Authenticated Key Exchange**

DH answers:
"Can we establish the same secret?"

Authentication answers:
"Who did I establish it with?"

Digital signatures + certificates

Pre-shared key (PSK)

Protocol-specific authentication

Identity + **DH Exchange** + **Handshake Context** → **Authenticated / Verified Exchange**

Real protocols bind the peer's identity and relevant handshake/transcript information to the key exchange.



Mallory can generate a valid DH value: $M = g^m \bmod p$

Valid DH value ≠ authenticated as Bob

If Mallory substitutes her own DH value without Bob's valid authentication credential:
Authentication verification FAILS

Secure key agreement requires both a shared secret and authentication of the peer/exchange.

AUTHENTICATED DIFFIE–HELLMAN

How Can We Authenticate a DH Exchange?

DH → establish shared secret

Authentication → establish peer identity

Together → AUTHENTICATED KEY EXCHANGE

1. CERTIFICATE + DIGITAL SIGNATURE

BEFORE / LONG-TERM CREDENTIAL

Bob already has: $SK_B + Cert_B$

Certificate binds Bob's identity to PK_B.

$A = g^a \quad B = g^b$

$Sig_B = Sign_{SKB}(\text{Handshake Context})$

Validate $Cert_B$

Trust / obtain PK_B

$Verify_{PKB}(\text{Handshake Context}, Sig_B)$

✓ VALID — Bob authenticated

Mallory can create her own DH value, but not Bob's valid signature without SK_B.

Exactly what is signed is protocol-defined.

2. PRE-SHARED KEY (PSK)

BEFORE THIS DH EXCHANGE

Alice ↔ PSK_{AB} ↔ Bob

PSK_{AB} = secret shared beforehand

$A = g^a, \quad B = g^b$

DH establishes a NEW shared secret:

$Z = g^{ab}$

PSK_{AB} + Handshake Context → Auth Proof

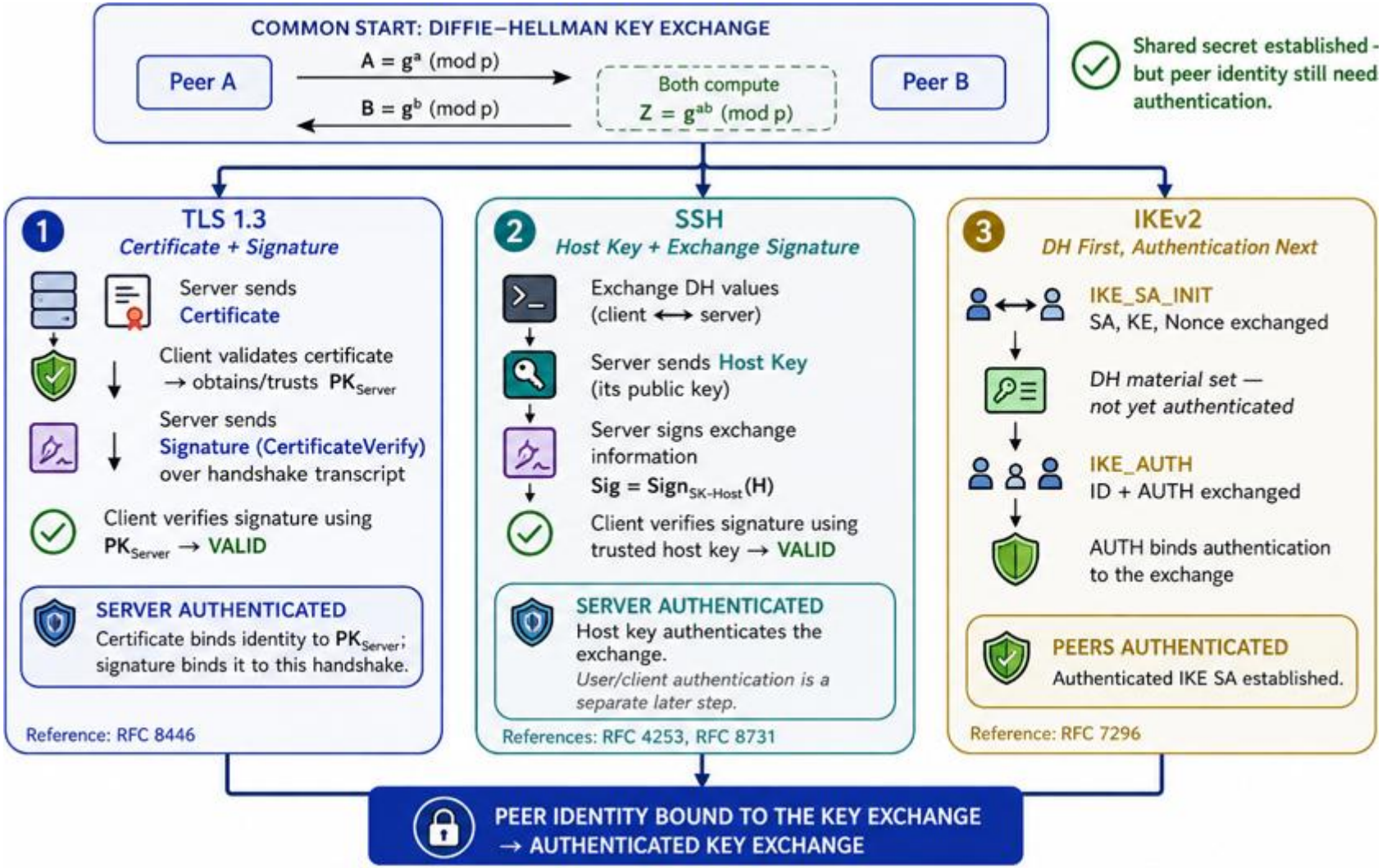
Only a peer holding PSK_{AB} can prove or verify this.

PSK_{AB}	**DH secret Z**
Before exchange — auth credential	*During exchange — fresh shared secret*

$PSK_{AB} \neq Z$

DH creates the new shared secret; authentication comes from a credential the peer already possesses — such as a signing key/certificate or a pre-shared secret.

How Do TLS, SSH and IKEv2 Authenticate the Key Exchange?



Different protocols authenticate different handshake structures, but the common goal is to bind peer identity to the key exchange.

If Bob's Long-Term Authentication Key Is Compromised Later, What About Earlier Sessions?

EARLIER — SESSION 1

$a_1 \rightarrow A_1 = g^{a1} \bmod p$ $\qquad$ $b_1 \rightarrow B_1 = g^{b1} \bmod p$

$$Z_1 = B_1^{a1} = A_1^{b1} = g^{a1b1} \bmod p$$

Long-term authentication key SK_B
Authenticates Bob and the exchange

Ephemeral DH values a_1, b_1
Establish the shared secret Z_1

After exchange: a_1, b_1 → ERASED

Attacker may have recorded: p, g, A_1, B_1, encrypted traffic

LATER: SK_B → COMPROMISED

PAST SESSION

Attacker still has only the recorded public DH values, but not a_1 or b_1.

$A_1 = g^{a1} \bmod p$ $\quad$ $B_1 = g^{b1} \bmod p$

Z_1 cannot feasibly be reconstructed from these recorded values.

FORWARD SECRECY

FUTURE SESSIONS

With compromised SK_B, Mallory may authenticate as Bob in a new exchange.

Forward secrecy protects past sessions; it does not make a compromised credential safe for future use.

Fresh ephemeral DH secrets protect past sessions from later compromise of long-term authentication credentials; compromised credentials can still threaten future authentication.

FROM DH TO DATA PROTECTION

Can the Shared Secret Z Become the AES or ChaCha Key?

$$Z = g^{ab} \bmod p$$

DH has established a shared secret. What key material does the secure session actually need?

WHY A KDF?

1. Required key size / form

Not automatically the key size a cipher needs.

AES-128 → 128-bit key

AES-256 → 256-bit key

ChaCha20 → 256-bit key

2. Multiple session keys

One secret can derive keys per direction, e.g.:

$K_{Alice \to Bob}$

$K_{Bob \to Alice}$ **+ other key material**

3. Context binding

KDF binds context and handshake info to the session.

DH Exchange → **Shared Secret** → **KDF** → **Session Keys** → **AEAD** → **Protected Data**

KDF: e.g., HKDF · AEAD: AES-GCM / ChaCha20-Poly1305

Z → **PRK** → **Required Key Material**

HKDF-Extract *HKDF-Expand (context/info)*

KEY AGREEMENT → KEY DERIVATION → DATA PROTECTION

DH → KDF → AEAD

DH establishes a shared secret; a KDF derives the context-specific session key material required for data protection.

WHAT SHOULD I REMEMBER?

From Diffie–Hellman Mathematics to a Secure Session

1 HARD MATHEMATICAL PROBLEM — $x \rightarrow g^x \bmod p$ (easy) | $g^x \bmod p \rightarrow x$ (hard)

Easy forward — hard reverse.

2 KEY AGREEMENT — $A = g^a$, $B = g^b$ — $Z = g^{ab}$

Same shared secret — never transmitted.

3 AUTHENTICATE THE EXCHANGE — MITM possible — DH + Auth → Authenticated Exchange

Know who the peer is.

4 FRESH EPHEMERAL DH — $a_i, b_i \rightarrow$ discard — FORWARD SECRECY

Later credential compromise need not expose earlier sessions.

5 DERIVE & USE KEYS — Z → KDF → Session Key Material → AEAD

Protect the actual data.

Applied in: TLS · SSH · IKEv2/IPsec

- **DH is key agreement — not encryption.**
- **Authenticated ephemeral DH can provide forward secrecy.**
- **Unauthenticated DH is vulnerable to MITM.**
- **The shared secret feeds a KDF; derived keys protect data.**

Practical DH security comes from the complete construction: strong groups + authentication + fresh ephemeral values + key derivation.

REFERENCES

References & Standards

FOUNDATION

Aumasson, J.-P., Serious Cryptography, 2nd edition: Chapter 11 — Diffie–Hellman

Paar, C. & Pelzl, J., Understanding Cryptography, 2nd edition: Chapter 8, specifically §8.1 — Diffie–Hellman Key Exchange

Stallings, W., Cryptography and Network Security 8th edition: Chapter 10, Specifically 10.1 – Diffie-Hellman Key Exchange

Diffie, W. & Hellman, M. E. "New Directions in Cryptography," IEEE Transactions on Information Theory, 1976.

DH GROUPS & CURVES

RFC 7919
Negotiated Finite Field DH Ephemeral Parameters for TLS — FFDHE groups.

RFC 7748
Elliptic Curves for Security — X25519 / X448.

RFC 3526
More MODP Diffie–Hellman Groups for IKE — MODP groups.

AUTHENTICATED PROTOCOLS

RFC 8446
The Transport Layer Security (TLS) Protocol Version 1.3.

RFC 4253
The Secure Shell (SSH) Transport Layer Protocol — exchange hash / server host-key authentication.

RFC 8731
SSH Key Exchange Method Using Curve25519 and Curve448.

RFC 7296
Internet Key Exchange Protocol Version 2 (IKEv2).

KEY DERIVATION

RFC 5869
HMAC-based Extract-and-Expand Key Derivation Function (HKDF).